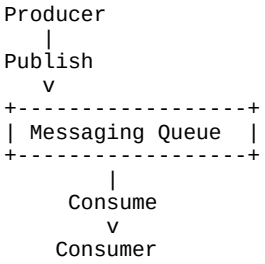
SDE-1 High-Level Design Study Guide
Distributed Messaging Queue

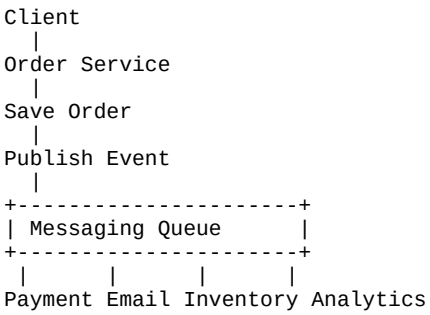
1. What is a Messaging Queue?

Architecture



2. Why is it Needed?

Real-world Order Processing

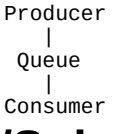


3. Messaging Patterns

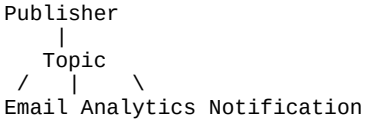
Point-to-Point:
Producer -> Queue -> One consumer receives each message.
Used for order processing, background jobs.

Publish/Subscribe:
Publisher -> Topic -> Multiple subscribers.
Every subscriber receives a copy.
Used for notifications, analytics and event-driven systems.

Point-to-Point Architecture

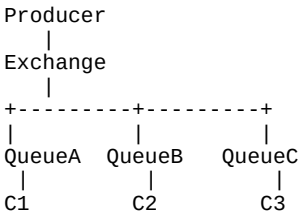


Pub/Sub Architecture



4. RabbitMQ

RabbitMQ Architecture



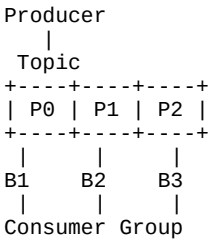
RabbitMQ Components

Exchange Types

Direct: exact routing key.
Fanout: broadcast to all queues.
Topic: wildcard routing (order.*).
Headers: route using headers.

5. Kafka

Kafka Architecture



Kafka Components

6. Failure Handling

Queue Full:
- Reject producers
- Block producers
- Drop/expire messages
- Dead Letter Queue
- Scale with partitions

Queue Down:
- Single broker: unavailable until recovery.
- Distributed brokers: replicas elect a new leader.

Consumer Down:
- No ACK received.
- Message is re-delivered to another consumer.

Consumer Cannot Process:
- Retry several times.
- Move to Dead Letter Queue (DLQ) after retry limit.

7. RabbitMQ vs Kafka

RabbitMQ:
- Queue-based

- Deletes after ACK
- Excellent for task queues
- Rich routing using exchanges

Kafka:
- Log-based
- Retains messages
- Excellent for event streaming
- High throughput via partitions

Interview Questions & Detailed Answers

Q1. Why use a messaging queue?
A. It decouples services, enables asynchronous processing, improves latency, reliability and scalability.

Q2. Difference between synchronous and asynchronous communication?
A. Synchronous waits for a response before continuing. Asynchronous publishes work and continues immediately.

Q3. What happens if a consumer crashes?
A. The broker waits for an ACK. Without an ACK, the message is re-delivered after timeout or rebalance.

Q4. Why is ACK required?
A. ACK confirms successful processing. Only then is the message removed, preventing message loss.

Q5. What is a Dead Letter Queue?
A. A queue that stores messages which repeatedly fail processing after configured retries.

Q6. How do multiple consumers avoid duplicate work?
A. In point-to-point systems, the broker assigns each message to only one consumer in a consumer group.

Q7. Why are partitions used?
A. Partitions allow parallel processing and horizontal scaling across multiple brokers.

Q8. How does a distributed queue survive failures?
A. Data is replicated across brokers. If a leader fails, another replica becomes leader.

Q9. Queue vs Pub/Sub?
A. Queue delivers each message to one consumer. Pub/Sub broadcasts to every subscriber.

Q10. How do you scale to millions of messages?
A. Increase partitions, brokers and consumers; replicate data; batch operations; optimize storage.